



PYTHON NOTES — PYTHON DICTIONARIES AND TUPLES



TOPIC: DICTIONARIES (PART 2)

11. LOOPING THROUGH A DICTIONARY

A dictionary can contain many pieces of information.

Instead of printing every item individually, we can use loops.

Python provides several ways to loop through dictionaries.

12. USING `.items()`

The `.items()` method returns both the **key** and the **value** together.

General syntax:

```
dictionary.items()
```

Example:

```
student = {  
    'name': 'Christabel',  
    'age': 14,  
    'school': 'Silver Fountain College'  
}  
  
for key, value in student.items():  
    print(key, ': ', value)
```

Output:

```
name : Christabel  
age : 14  
school : Silver Fountain College
```

Explanation

During every loop:

```
key
```

stores the dictionary key.

```
value
```

stores the corresponding value.

This is the method we used most often in our projects.

13. USING `.keys()`

Sometimes we only need the keys.

Example:

```
student = {  
    'name': 'Christabel',  
    'age': 14,  
    'school': 'Silver Fountain College'  
}  
  
for key in student.keys():  
    print(key)
```

Output:

```
name  
age  
school
```

14. USING `.values()`

Sometimes we only need the values.

Example:

```
student = {  
    'name': 'Christabel',  
    'age': 14,  
    'school': 'Silver Fountain College'  
}  
  
for value in student.values():  
    print(value)
```

Output:

```
Christabel  
14  
Silver Fountain College
```

15. `.items()` VS `.keys()` VS `.values()`

`.items()`

Returns:

```
key + value
```

Example:

```
for key, value in student.items():
```

`.keys()`

Returns:

only the keys

Example:

```
for key in student.keys():
```

```
.values()
```

Returns:

only the values

Example:

```
for value in student.values():
```

16. NESTED DICTIONARIES

A dictionary can contain another dictionary.

Example:

```
students = {  
    'student1': {  
        'name': 'Christabel',  
        'age': 14  
    },  
    'student2': {  
        'name': 'Daniel',  
        'age': 15  
    }  
}
```

This is called a **nested dictionary**.

The outer dictionary stores multiple students.

Each student is another dictionary.

17. ACCESSING NESTED DICTIONARIES

Example:

```
print(students['student1'])
```

Output:

```
{  
  'name': 'Christabel',  
  'age': 14  
}
```

To access a specific value:

```
print(students['student1']['name'])
```

Output:

```
Christabel
```

Python first finds:

```
student1
```

Then inside that dictionary it finds:

```
name
```

18. LOOPING THROUGH A NESTED DICTIONARY

Example:

```
for key, value in students.items():
    print(key)

    for user_key, user_value in value.items():
        print(user_key, ': ', user_value)
```

Output:

```
student1
name : Christabel
age : 14

student2
name : Daniel
age : 15
```

Notice the **two loops**.

The first loop moves through the outer dictionary.

The second loop moves through each student's dictionary.

19. OUR STUDENT MANAGEMENT SYSTEM PROJECT

Our Student Management System was built using nested dictionaries.

Structure:

```
students = {
    'student1': {
        ...
    },
    'student2': {
        ...
    }
}
```

Each student stored:

- name

- age
- school
- favorite_subject
- hobby

This project helped us practice almost every dictionary concept we learned.

20. DISPLAYING ALL STUDENTS

We used:

```
for key, value in students.items():
```

to move through every student.

Then:

```
for user_key, user_value in value.items():
```

displayed every piece of information about that student.

This is one of the most useful applications of nested dictionaries.

21. VIEWING ONE STUDENT

The program asked:

```
Which student's information do you want to view?
```

Python then checked:

```
if user_input in students:
```

If the student existed:

```
selected_student = students[user_input]
```

We then displayed the information using another loop.

22. UPDATING A STUDENT

The update process followed these steps:

1. Ask which student.
2. Check whether the student exists.
3. Ask which piece of information to update.
4. Check whether that key exists.
5. Ask for the new value.
6. Replace the old value.
7. Display the updated student.

This taught us that dictionaries allow values to be modified very easily.

23. ADDING A NEW STUDENT

We allowed the user to create a completely new dictionary.

Example:

```
students[new_student] = {  
    ...  
}
```

Python created an entirely new nested dictionary inside the existing one.

24. DELETING A STUDENT

We used:


```
students.pop(student_id)
```

This removed the entire student record.

25. DELETING A KEY

Example:

```
selected_student.pop(key_to_delete)
```

This removed only one key and its corresponding value.

Example:

Before:

```
name  
age  
school  
hobby
```

After removing hobby:

```
name  
age  
school
```

End of Part 2

In Part 3 we'll finish with:

- Best practices
- Common mistakes
- Why dictionaries are powerful
- Real-world applications
- Complete progress tracker
- Summary
- "Next: Tuples"